KIROVAULT

API Contracts & Implementation Tasks

A practical command contract and staged task breakdown for handing implementation to a developer or coding agent.

DOCUMENT PURPOSE

This standalone companion document complements the KiroVault Desktop App Blueprint. It follows the Windows-first, local-first product direction. Recommendations are proposed guidance, not claims that these features already exist.

Contents and implementation guidance

Command contract pattern

- Request: typed arguments, explicit paths and operation options.
- Validation: Rust checks path, access, scope, collision and current filesystem state.
- Response: structured status, payload, warnings and per-item outcomes.
- Errors: stable codes such as PermissionDenied, NotFound, Conflict, Unsupported, Cancelled and PartialFailure.
- Do not expose raw filesystem access to arbitrary UI components.

Initial command groups

- filesystem.list_directory(path, sort, filters) → entries + metadata + warnings.
- filesystem.rename(path, new_name) → final path or conflict result.
- filesystem.copy/move(items, destination, conflict_policy) → per-item outcomes and progress id.
- filesystem.delete(items, mode) → confirmation requirement and per-item results.
- indexing.start(roots, exclusions), pause, resume, cancel, progress.
- thumbnails.request(file_id/path key, target_size), cancel and cache status.
- search.query(filter model) → result records with original paths.

Implementation backlog

- P0: scaffold app, tokens, navigation shell, typed command wrapper and error boundary.
- P1: real directory listing, path navigation, safe operations and operation states.
- P2: image thumbnails/viewer, video poster/playback, PDF and text/code preview.
- P3: SQLite schema, incremental scan, Recent/Favorites and smart collections.
- P4: storage insights, duplicate candidates, advanced formats and optional content search.
- P5: stress testing, accessibility, licensing, installer and release hardening.

Definition of done per task

- Acceptance criteria and error states documented.
- Unit/integration tests cover success, failure, cancellation and collision paths.
- UI reports progress and partial failure accurately.
- No preview executes content or silently mutates originals.
- Documentation and source references updated.

Example error response

- status: partial_failure; completed: 8; failed: 2; errors: [{path, code, message}].
- Keep the schema consistent across commands; never expose sensitive content in error messages.

References

- Tauri commands: https://v2.tauri.app/develop/calling-rust/
- Tauri security: https://v2.tauri.app/security/
- Serde serialization: https://serde.rs/
- SQLite docs: https://www.sqlite.org/docs.html
- KiroVault Desktop App Blueprint, pp. 18–23 (user-provided source).

Reference note: external documentation may change. Verify current versions, licensing, APIs and platform prerequisites during implementation.